Like the be_within matcher, be_between supports a fluent interface. You can use be_between(x, y).inclusive or be_between(x, y).exclusive to explicitly choose an inclusive or exclusive range. The default is inclusive, like Ruby's Comparable#between?(x, y).

Finally, both of these matchers have aliases that are designed to read well in composed matcher expressions: be_within is aliased to a_value_within and be_between is aliased to a_value_between.

Dynamic Predicates

A *predicate* is a method that answers a question with a Boolean answer. In Ruby, predicate method names typically omit the verb and end with a question mark. For example, Ruby's Array class provides an empty? method rather than is_empty.

This convention is so common that RSpec includes special support for it in the form of *dynamic predicate matchers*.

How to Use Them

When you use an unrecognized matcher of the form be_..., RSpec strips off the be_, adds a question mark to the end, and calls that method on the subject.

For instance, for RSpec to call array.empty?, you can use be_empty in your expectation:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb
```
expect([]).to be_empty
```

If empty? returns a truthy value (see *Truthiness*, on page 191), the expectation passes. You can alternately use a be_a_ or be_an_ prefix for predicates that are nouns. For example, if you had a user object with an admin? predicate, either of these would work:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb
```
expect(user).to be_admin
expect(user).to be_an_admin
```

The latter reads much more nicely. For predicates that begin with has_, such as hash.has_key?(:age), you can use a dynamic predicate matcher starting with have_:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb
```
hash = { name: 'Harry Potter', age: 17, house: 'Gryffindor' }
expect(hash).to have_key(:age)
```